Cloud Computing (CS3081): Mini Project Report

Name: Arjeet Singh | **Roll Number:** 230101078
Course: Cloud Computing (CS3081)
Repository: `https://github.com/rajeetvk/cloud-distillation_CS3081`

November 2026

Introduction and Architecture Overview

Modern conversational voice interfaces require extracting user intent and semantic slot values (e.g., date, time, location) with deterministic latency and strict adherence to schemas. While large language models (LLMs) handle zero-shot intent parsing with high accuracy, running them at the edge or in tight cloud containers is impractical due to high memory requirements and CPU inference bottlenecks.

To overcome this, our project decouples large-scale teacher inference from runtime serving through sequence-level knowledge distillation, containerization, and cloud-native orchestration.

System Architecture

Our system consists of five decoupled tiers running on local cloud infrastructure:

1. **Object Storage Tier**: SeaweedFS deployed in the `data` namespace, serving S3-compatible endpoints on port 8333 and filer interfaces on port 8888. It acts as our centralized object store for raw datasets, cleaned corpora, and trained model artifacts.
2. **Parallel Generation Tier**: Independent batch worker jobs running in the `pipeline` namespace, executing deterministic modulo sharding against an external Ollama teacher instance.
3. **Training and Distillation Tier**: A standalone PyTorch causal language modeling workflow executing sequence-level distillation on CPU hardware with prompt masking.
4. **Inference Service Tier**: A FastAPI application running in the `app` namespace, serving synchronous inference on port 8000 via background worker threads, complete with runtime JSON schema validation and dynamic S3 model weight retrieval.
5. **Ingress and Networking Tier**: A Traefik Ingress controller routing external port 8080 traffic to healthy pods, governed by stateless Network Access Control List (NACL) rules.

Task 1: Object Storage and Parallel Teacher Labelling

Storage Setup and Modulo Sharding

For local object storage, we deployed SeaweedFS inside the `data` namespace. During our initial configuration, we observed that SeaweedFS default master limits (`volume.max=5`) quickly exhausted writable volumes when separate collections were created for buckets. We resolved this by explicitly configuring `-volume.max=100` and provisioning buckets `datasets` and `models`. The raw pool of 3,000 unlabelled commands from `data/commands_pool.jsonl` was stored in bucket `datasets`.

To annotate 3,000 commands efficiently on a single laptop without race conditions or overlapping writes, we implemented a deterministic modulo sharding strategy across two concurrent worker pods (`worker-shard-0` and `worker-shard-1`):

$$\text{Line Number } n \pmod{\text{SHARD_COUNT}} = \text{SHARD_INDEX}$$

Shard 0 processed all even lines (indices $0, 2, \ldots, 2998$), while Shard 1 processed all odd lines (indices $1, 3, \ldots, 2999$). Each worker formatted user queries with our fixed system prompt from `prompts/teacher_prompt.txt`, queried the local `Phi-3.5-mini-instruct` teacher model with temperature $= 0.0$ and seed $= 42$, and streamed raw completions to `shard-0.jsonl` and `shard-1.jsonl`.

Data Cleaning and Mathematical Reconciliation

Raw outputs generated by the teacher exhibited occasional hallucinated slot values and duplicate commands. We implemented `worker/clean.py` to enforce five strict validation checks:

- **(a) Invalid JSON**: Completions that could not be parsed by Python's JSON decoder.
- **(b) Invalid Keys**: Objects that did not match the exact key set $\{\text{intent}, \text{date}, \text{time}, \text{place}\}$.
- **(c) Invalid Intent**: Output intent not belonging to the eight canonical classes.
- **(d) Grounding Violation**: Non-null slot values that did not appear as exact substrings of the input utterance.
- **(e) Duplicates**: Duplicate command texts, retaining only the entry with the lowest line index.

Table 1: Dataset Cleaning and Reconciliation Breakdown

Category	**Record Count**
Total Input Records from Raw Pool	3,000
Rejected (a): Malformed JSON	0
Rejected (b): Invalid Schema Keys	1
Rejected (c): Unknown Intent Class	14
Rejected (d): Ungrounded / Hallucinated Entity Slots	74
Rejected (e): Duplicate Command Texts	50
Cleaned Records Kept for Distillation	2,861
Mathematical Sum Check	$0+1+14+74+50+2,861 = 3,000$ (PASS)
SHA-256 Checksum of `train.jsonl`	`b2819ba22122...63b8`

As shown in Table 1, exactly 2,861 valid records were preserved. The cleaned dataset and its SHA-256 hash were saved to `datasets/v1/train.jsonl` and synced to `s3://datasets/v1/train.jsonl`.

Task 2: Student Model Training and Validation Scoring

Why Sequence-Level Distillation?

Knowledge distillation can be performed either at the logit level (matching probability distributions over the vocabulary) or at the sequence level (training the student on discrete text sequences generated by the teacher). In our project, logit-based distillation was mathematically and practically impossible:

1. **Vocabulary Mismatch**: The teacher model (`Phi-3.5`) employs a 32,064-token SentencePiece vocabulary, whereas the student (`Pythia-70M`) uses a 50,304-token GPT-NeoX byte-pair encoding (BPE) vocabulary. Their logits cannot be mapped without learning auxiliary projection adapters.
2. **Black-Box Inference API**: Ollama serves models via an inference endpoint that returns generated text strings rather than internal logits or hidden activations.

Consequently, we used sequence-level distillation (Kim & Rush, 2016), treating the teacher as an automated annotator and training the student causal language model with prompt masking:

$$\mathscr{L}_{\text{distill}} = -\sum_{t=1}^{T} \log P_{\text{student}}\left(y_t \mid y_{<t}, x\right)$$

where x represents the input prompt tokens and y represents the target JSON tokens. Prompt tokens were assigned a label of -100 so the student only computed loss over the JSON completion.

Subword Tokenizer Merges and Debugging

During initial training experiments, we discovered that student completions emitted truncated strings missing the opening brace (e.g., `intent": "alarm_set", ...`). Investigating the root cause revealed that the GPT-NeoX tokenizer merges whitespace with punctuation (e.g., `␣{"` is tokenized as a single subword). When slicing prompt

strings, the boundary split this combined token. We resolved this by tokenizing the prompt and JSON target strings separately into integer ID arrays before concatenation:

$$\text{input_ids} = \text{prompt_ids} + \text{target_ids}, \quad \text{labels} = [-100] \times |\text{prompt_ids}| + \text{target_ids}$$

This ensured that the entire JSON payload from the opening brace { to the closing brace } was trained with full supervision.

Training Dynamics and Empirical Evaluation

We trained on the first 1,500 records of the cleaned dataset using PyTorch with the AdamW optimizer ($\text{lr} = 5 \times 10^{-4}$, batch_size = 8, max_length = 128) entirely on CPU. We trained two model variants:

- **Student v1 (3 epochs)**: Cross-entropy loss steadily decreased from 1.0843 to 0.3365 to 0.1287.
- **Student v2 (4 epochs)**: Continued convergence from 0.9809 down to 0.0720 on epoch 4.

Table 2: Model Evaluation Comparison on Validation Dataset (100 Samples)

Evaluation Metric	Teacher (Phi-3.5)	Untrained Pythia	Student v1 (3 ep)	Student v2 (4 ep)
JSON Validity (%)	98.0%	0.0%	99.0%	100.0%
Intent Accuracy (%)	87.0%	0.0%	81.0%	81.0%
Date Slot Accuracy (%)	79.0%	0.0%	68.0%	76.0%
Time Slot Accuracy (%)	80.0%	0.0%	82.0%	81.0%
Place Slot Accuracy (%)	75.0%	0.0%	70.0%	74.0%
Full-Frame Accuracy (%)	42.0%	0.0%	30.0%	34.0%
Missing Predictions	0	0	0	0

As shown in Table 2, Student v1 achieved 99.0% JSON validity and 30.0% full-frame accuracy, satisfying the baseline requirements. Student v2 reached 100.0% JSON validity and 34.0% full-frame accuracy, comfortably exceeding the required half-teacher threshold ($\geq 21.0\%$).

Task 3: The Inference Service in a Container

Service Architecture and Event-Loop Optimization

The inference service is implemented using FastAPI in `service/main.py`. It provides:

- `POST /parse`: Receives JSON payload {`"text": "..."`}, formats the command, generates greedy autoregressive tokens on CPU, validates the output against `data/schema.json`, and returns HTTP 200 with response headers `X-Model-Version` and `X-Served-By`. If the model outputs malformed JSON, it returns HTTP 500 with {`"error": "invalid_model_output"`}.
- `GET /healthz`: Non-blocking liveness probe returning HTTP 200.
- `GET /readyz`: Readiness probe returning HTTP 503 until model weights are loaded from S3, then HTTP 200.

Critical Event-Loop Bug Fix: Initially, `POST /parse` was defined as an asynchronous function (`async def parse`). In FastAPI, async route handlers run on the main asyncio event loop. Because `model.generate()` is a CPU-bound PyTorch calculation taking 3 to 5 seconds, the main event loop was completely blocked during inference. This starved the liveness probe endpoint (`/healthz`), causing Kubernetes to report timeouts and restart the container in a CrashLoopBackOff state. We resolved this by changing the route definition to a synchronous function (`def parse`). FastAPI then automatically dispatches requests to an internal threadpool, keeping the main asyncio loop free to respond to health probes in < 1 ms.

Container Optimization and Resource Benchmarks

To respect the ≤ 2 GB image ceiling, we installed CPU-only PyTorch wheels (`torch==2.5.1+cpu`), avoiding over 2.5 GB of extraneous CUDA GPU binaries. The resulting Docker image content size was only 345 MB (1.63 GB uncompressed). Model weights were excluded from the image layers and fetched dynamically from SeaweedFS S3 at startup.

Under strict hardware constraints (1 CPU core, 1 GiB RAM), we dispatched 50 sequential requests to `POST /parse`. The results, logged in `results/benchmark_50_requests.json`, confirmed:

- **Success Rate**: 50 out of 50 requests succeeded (100.0% success, 0 OOM kills).
- **Memory Utilization**: Peak RAM was 374.5 MiB, well within the 1,024 MiB limit.
- **Latency Metrics**: Mean latency was 5,624 ms, median (p_{50}) was 5,599 ms, and 95th percentile (p_{95}) was 7,001 ms.
- **Fault Injection**: When executed with `FAULT_INJECT=1`, the service returned HTTP 500 with body {`"error": "invalid_model_output"`} and preserved response headers.

Task 4: Kubernetes Deployment, Scaling, and Reliability

Cluster Configuration and Connection Draining

The production deployment in `cluster/app.yaml` runs in the `app` namespace with two replicas. Ingress is managed by Traefik on port 8080. S3 credentials are provided via Kubernetes Secrets. Resources are capped at 250m CPU / 256Mi RAM requests and 1000m CPU / 1024Mi RAM limits.
Graceful Connection Draining: During rolling update tests, terminating pods were initially dropped abruptly by Kubernetes, causing in-flight requests routed by Traefik to fail with connection reset errors. We resolved this by adding a `preStop` lifecycle hook with `sleep 5` and configuring `maxSurge: 1` and `maxUnavailable: 0`. This 5-second buffer gave the Traefik Ingress controller time to de-register terminating pod endpoints before SIGTERM was issued.

Empirical Cluster Reliability Results

1. **Cold Start Benchmark**: We tested pod cold starts by deleting running pods across three runs. Each new pod scheduled, initialized, downloaded model weights from SeaweedFS S3, and became Ready in 6.71 s, 6.69 s, and 6.84 s, establishing a median cold-start time of 6.71 s (`results/cold_start_results.json`).
2. **Ingress Load Balancing**: We ran a 2-minute headless Locust load test with 4 concurrent users. Across 57 successful responses, Replica 1 (`gxmpm`) served 29 requests (50.88%) and Replica 2 (`vftlx`) served 28 requests (49.12%). Both replicas comfortably exceeded the $\geq 30\%$ requirement, confirming balanced round-robin routing.
3. **Zero-Downtime Rolling Update**: We updated the deployment from Student v1 to Student v2 under traffic, followed by an immediate rollback to v1. Across 10 active probe requests, zero failures occurred (0.0% failure rate, satisfying the $\leq 0.5\%$ target), with headers confirming traffic reverted to `X-Model-Version: v1`.
4. **Self-Healing Recovery**: Under continuous traffic, a running pod was forcefully deleted. Kubernetes self-healed and restored the 2-replica state in 6.74 s with 0.0% dropped requests (beating the $\leq 2.0\%$ threshold).

Task 5: Subnetting and Stateless Network Access Control (NACL)

Subnet Allocation Plan

Given the private CIDR block `10.20.0.0/16`, we designed a structured subnet topology in descending order of host requirements:

Table 3: Subnet Allocation Table for 10.20.0.0/16

Subnet Name	Prefix	Network Address	Usable IP Range	Broadcast Address	Usable Hosts
`app`	/22	`10.20.0.0`	`10.20.0.1` – `10.20.3.254`	`10.20.3.255`	1,022
`public`	/24	`10.20.4.0`	`10.20.4.1` – `10.20.4.254`	`10.20.4.255`	254
`data`	/25	`10.20.5.0`	`10.20.5.1` – `10.20.5.126`	`10.20.5.127`	126
`management`	/27	`10.20.5.128`	`10.20.5.129` – `10.20.5.158`	`10.20.5.159`	30

Stateless NACL Rules and Simulation

Unlike stateful security groups, stateless NACLs do not track connection states, requiring explicit bidirectional rules:

- **Ephemeral Port Return Traffic**: Inbound HTTP requests on ports 80/443 originate from dynamic client ephemeral ports (1024–65535). The outbound NACL must explicitly permit traffic returning to those ephemeral ports. Similarly, load balancer connections to the app subnet on port 8000 require an inbound rule permitting response traffic on ports 1024–65535 from `10.20.0.0/22`.
- **First-Match Evaluation**: NACLs evaluate rules in ascending order, terminating on the first match. To block the abusive subnet `198.51.100.0/24`, Rule 100 must be an explicit `DENY` placed before Rule 110/120 `ALLOW` rules.

We verified our rule set against the reference packet vectors using `tools/nacl_sim.py` with the `--check` flag, achieving a verified pass rate of 6/6 (100.0%).

Task 6: Reproducibility, Testing, and CI/CD Pipeline

Our GitHub Actions pipeline (`.github/workflows/ci.yml`) executes five sequential stages:

1. **Secret Scanning**: Scans the repository using Gitleaks to ensure no private keys or credentials are committed (zero findings).
2. **Linting**: Validates PEP8 compliance and syntax with Flake8.
3. **Unit Tests**: Executes six unit tests in `tests/test_service.py` covering schema validation, health probes, and fault injection.
4. **Model Evaluation**: Evaluates model predictions against `data/validation.jsonl`.
5. **Quality Gate**: Enforces `score.py --min-validity 95`.

Quality Gate Verification: When evaluated with the untrained base model (achieving 0.0% JSON validity), the quality gate caught the defect and failed the build with a red indicator. When evaluated with our distilled student model (achieving 99.0% JSON validity), the pipeline passed with a green indicator, demonstrating that the gate operates as intended.

Task 7: Performance, Cost Estimation, and SRE Review

Cost Estimation Model

Using the hypothetical rates defined in `data/rate_card.json` (3.50 INR per vCPU-hour and 0.45 INR per GiB-hour), each replica is billed for its configured resource ceilings (1 vCPU, 1 GiB):

$$\text{Hourly Cost per Replica} = (1.0 \times 3.50) + (1.0 \times 0.45) = 3.95 \text{ INR/hr}$$

$$\text{Total Hourly Cost for 2 Replicas} = 2 \times 3.95 = 7.90 \text{ INR/hr}$$

During our 2-minute Locust load test (57 requests served), the extrapolated throughput was:

$$\text{Requests per Hour} = \frac{57 \text{ requests}}{2 \text{ minutes}} \times 60 \text{ minutes} = 1,710 \text{ requests/hour}$$

$$\text{Cost per 1,000 Requests} = \left(\frac{7.90}{1,710}\right) \times 1,000 = 4.62 \text{ INR}$$

Table 4: Inference Cost and Throughput Projections

Operating Scenario	Hourly Throughput	Total Cost / Hour	Cost per 1,000 Requests
Measured 2-Minute Locust Test	1,710 req/hr	7.90 INR	4.62 INR
Continuous Sequential Load (2 Cores)	1,222 req/hr	7.90 INR	6.46 INR
Single Replica Idle Baseline	0 req/hr	3.95 INR	N/A

Service Level Objective (SLO) and Error Budget

- **SLO Target**: At least 99.0% of all `POST /parse` requests must return HTTP 200 within 30 seconds.
- **Measurement Window**: One 2-minute load test run (57 requests, 4 concurrent users).
- **Error Budget**: Error Budget $= 57 \times (1.0 - 0.99) = 0.57$ requests.
- **Measured Result**: 57 requests succeeded out of 57 (100.0% availability). Zero errors occurred, consuming 0% of the error budget (100% budget remaining).

Comprehensive System Review Table

Table 5 summarizes our empirical observations, identified limitations, and proposed improvements across the six system components.

Table 5: Comprehensive System Review Table

Component	Evidence (Measured)	Limitation Found	Proposed Improvement
Data and storage	2,861 clean records reconciled ($3000 = 0+1+14+74+50+2861$) with SHA-256 hash	SeaweedFS default of `volume.max=5` caused volume exhaustion across collections	Configure `-volume.max=100` on master and pre-allocate volumes through filer
Model quality	Student v2 achieved 100.0% JSON validity and 34.0% full-frame accuracy (teacher 42.0%)	Exposure bias during autoregressive decoding causing slot drift on unseen tokens	Integrate context-free grammar decoding (Outlines/BNF) and DPO alignment
Latency and throughput	Locust test measured $p_{50} = 5,599$ ms, $p_{95} = 7,001$ ms, and throughput of 0.48 req/s	Full 32-bit float PyTorch CPU inference requires $\sim$5 s per request on CPU	Quantize weights to INT8/INT4 using ONNX Runtime or `llama.cpp` for $<$ 150 ms latency
Resource utilisation	Peak memory measured at 374.5 MiB against 1 GiB limit, 1.0 CPU limit adhered to, 0 OOM kills	Single container pod uses single-threaded PyTorch execution	Configure `torch.set_num_threads(2)` and use Prometheus custom metrics for HPA
Reliability and recovery	6.71 s median cold start, 0.0% failure rate during rolling update, pod recovery in 6.74 s	Pods marked terminating without `preStop` hooks dropped in-flight requests	Inject `preStop sleep 5` lifecycle hook and configure graceful termination periods
Security and cost	6/6 stateless NACL packet simulation pass (100%), estimated cost of 4.62 INR per 1,000 requests	Stateless NACL requires manual bidirectional ephemeral port rules (1024–65535)	Adopt state-aware security groups or Kubernetes NetworkPolicies with automated mTLS